Mini-Projet : Simulation d'une application de gestion avec Python

1. Contexte

Dans les applications web modernes (e-learning, gestion de clubs, inscriptions, etc.), différents utilisateurs interagissent avec un système selon leur rôle. Avant de développer avec Django, il est important de comprendre comment organiser les données, gérer l'authentification, implémenter les fonctionnalités et structurer un programme Python en modules.

Dans ce TP, vous allez créer une simulation d'application en Python en ligne de commande, afin de reproduire la logique d'une application web en utilisant les notions vues en cours : listes, dictionnaires, fonctions, conditions, boucles et modules.

À l'issue de ce TP, vous serez capables de manipuler des structures de données, créer des fonctions, organiser un programme, gérer les entrées utilisateur et implémenter une authentification simple. Vous développerez également des compétences en conception (logique fonctionnelle, menu interactif, opérations CRUD) et en méthodologie (organisation du projet, travail en équipe, documentation).

2. Principe du TP

Vous devez développer un programme Python simulant une application comportant plusieurs types d'acteurs. Selon le projet choisi, les rôles peuvent correspondre par exemple aux acteurs suivants :

Acteur	Exemple
Administrateur	Gestion globale
Utilisateur	Étudiant
Responsable	Enseignant / responsable club

Chaque acteur dispose d'un ensemble de fonctionnalités spécifiques accessibles après authentification.

2.1. Structure des données

Les utilisateurs doivent être stockés dans des structures de données Python composées de listes contenant des dictionnaires imbriqués. Une liste distincte doit être créée pour chaque type d'acteur.

Exemple de structure possible:

administrateurs = [{1: {"nom": "Admin1", "email": "admin@mail.com", "password": "123"}}]

etudiants = [{1: {"nom": "Ali", "email": "ali@mail.com", "password": "abc"}}]

enseignants = [{1: {"nom": "Sara", "email": "sara@mail.com", "password": "xyz"}}]

L'identifiant (id) doit être utilisé comme clé principale du dictionnaire représentant chaque utilisateur.

2.2. Fonctionnalités à implémenter

- **Fonction d'inscription :** Créer une fonction **inscription()** permettant de :
  - Ajouter un nouvel utilisateur
  - Choisir le type d'acteur
  - Générer un identifiant
  - Stocker les informations dans la liste correspondante

- **Fonction de connexion :** Créer une fonction **connexion(username, password)** qui :
  - Vérifie les informations dans les listes existantes
  - Identifie le type d'acteur
  - Affiche le menu correspondant

  Contraintes :
  - Maximum 3 tentatives de connexion
  - Message d'erreur si les informations sont incorrectes

2.3. Menus des fonctionnalités

Après authentification, un menu spécifique doit être affiché en fonction du type d'utilisateur connecté.

Menu Administrateur (exemple)

```
===== MENU ADMINISTRATEUR =====

1 Ajouter un utilisateur

2 Supprimer un utilisateur

3 Modifier les informations d'un utilisateur

4 Consulter la liste des utilisateurs

5 Rechercher un utilisateur

6 Déconnexion
```

Menu Utilisateur (exemple)

```
===== MENU UTILISATEUR =====

1 Consulter mes informations

2 Modifier mes informations

3 Rechercher une ressource

4 Déconnexion
```

Menu Responsable (exemple)

```
===== MENU RESPONSABLE =====

1 Ajouter une ressource

2 Modifier une ressource

3 Supprimer une ressource

4 Consulter les ressources

5 Déconnexion
```

- **Fonctions de gestion**

Vous devez implémenter différentes fonctions permettant de gérer les opérations suivantes : ajout, suppression, modification, consultation et recherche des données.

3. Organisation du projet

Le programme doit être structuré en plusieurs fichiers Python afin de favoriser la modularité et la lisibilité du code.

Fichier	Rôle
main.py	Programme principal
authentification.py	Inscription et connexion
admin.py	Fonctions administrateur
utilisateur.py	Fonctions utilisateur
responsable.py	Fonctions responsable
data.py	Stockage des listes

4. Livrables attendus

Chaque groupe doit remettre :

4.1. Code source :

Le programme doit être fonctionnel, correctement structuré et suffisamment commenté pour faciliter la compréhension du code.

4.2. Rapport (3 à 5 pages)

Le rapport doit contenir :

1. Une description du système choisi
2. Une description des acteurs
3. La structure des données utilisée
4. La description des fonctions principales
5. Des captures d'écran illustrant le fonctionnement du programme.

4.3. Démonstration orale

Chaque groupe devra présenter (en 5 à 7min) l'architecture de son programme, expliquer les principales fonctionnalités implémentées et démontrer le fonctionnement de l'application développée.